

Học Sâu cho Khoa Học Dữ liệu

Chapter 4. RNN

Hà Minh Tuấn

`hmtuan@hcmus.edu.vn`

Khoa Toán - Tin học

Trường Đại học Khoa học Tự nhiên, ĐHQG-HCM

Ngày 31 tháng 3 năm 2026

Nội dung bài học

1. giới thiệu
2. Recurrent Neural Network
3. ví dụ
4. Forward Pass
5. Backpropagation Through Time trong RNN
6. vanishing and exploding in RNN
7. ví cụ chạy code

Thử thách nhỏ: Đoán từ tiếp theo

“Hà Nội là _____ của Việt Nam.”

- Bạn đoán từ gì?

Thử thách nhỏ: Đoán từ tiếp theo

“Hà Nội là _____ của Việt Nam.”

- Bạn đoán từ gì?
- Hầu hết sẽ đoán: **“thủ đô”**

Thử thách nhỏ: Đoán từ tiếp theo

“Hà Nội là _____ của Việt Nam.”

- Bạn đoán từ gì?
- Hầu hết sẽ đoán: **“thủ đô”**
- Vì sao bạn đoán đúng?
 - ▶ Bạn đọc **tuần tự** từ trái sang phải
 - ▶ Mỗi từ đã đọc giúp bạn **thu hẹp khả năng** cho từ tiếp theo
 - ▶ Bộ não bạn **nhớ ngữ cảnh** đã đọc trước đó

Câu hỏi

Mạng neural có thể làm được điều tương tự không?

Dữ liệu tuần tự (Sequential Data) ở khắp nơi

- **Ngôn ngữ tự nhiên**

- ▶ Câu văn, đoạn hội thoại

- **Chuỗi thời gian**

- ▶ Giá cổ phiếu, nhiệt độ, nhịp tim

- **Âm thanh & Nhạc**

- ▶ Tín hiệu sóng âm theo thời gian

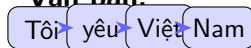
- **Video**

- ▶ Chuỗi các khung hình liên tiếp

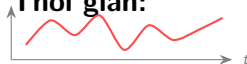
- **DNA**

- ▶ Chuỗi nucleotide A-T-G-C

Văn bản:



Thời gian:



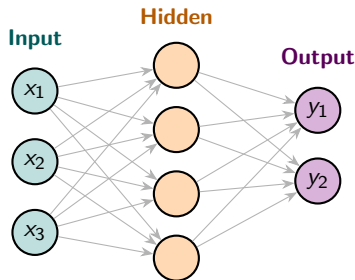
DNA:



Đặc điểm chung

Thứ tự rất quan trọng — đảo thứ tự sẽ thay đổi ý nghĩa!

Nhắc lại: Feedforward Neural Network



Đặc điểm:

- Tín hiệu đi **một chiều**: Input $\rightarrow$ Hidden $\rightarrow$ Output
- Mỗi mẫu dữ liệu xử lý **độc lập**
- **Không có bộ nhớ** giữa các lần xử lý

Phù hợp cho:

- Phân loại ảnh (một ảnh $\rightarrow$ một nhãn)
- Hồi quy (đầu vào cố định $\rightarrow$ đầu ra)

Feedforward Network “bắt lực” với dữ liệu tuần tự

Thử cho Feedforward xử lý câu:

“Hà Nội là _____ của Việt Nam.”

Vấn đề 1: Kích thước đầu vào cố định

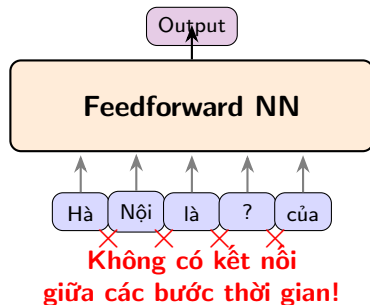
- Câu 5 từ $\neq$ câu 20 từ
- Feedforward cần vector đầu vào kích thước cố định

Vấn đề 2: Không nhớ ngữ cảnh

- Xử lý từ “của” mà **không biết** trước đó là “Hà Nội là”

Vấn đề 3: Không chia sẻ tham số

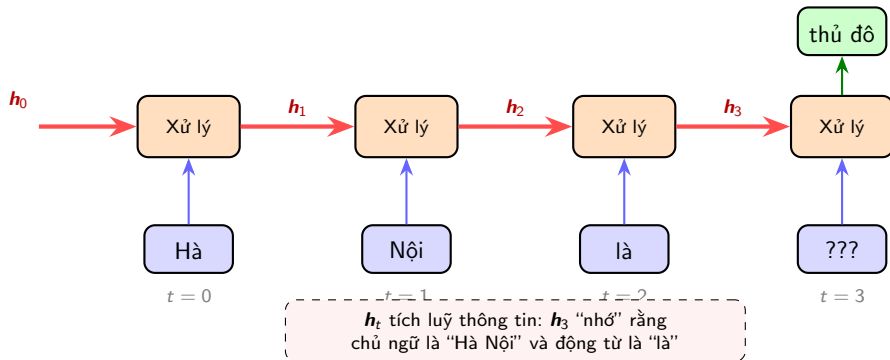
- Cùng từ “là” ở vị trí 3 và vị trí 7 dùng trọng số khác nhau



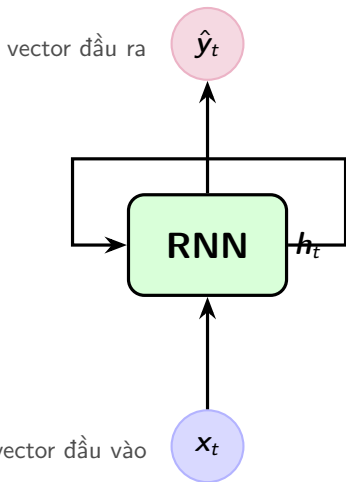
Ý tưởng cốt lõi: Mạng neural có “bộ nhớ”

Ý tưởng then chốt

Thêm một **trạng thái ẩn (hidden state)** h_t — một vector lưu lại thông tin từ **tất cả các bước trước đó**.



Mạng Neural Hồi quy (Recurrent Neural Networks — RNNs)



Áp dụng một **quan hệ truy hồi** (recurrence relation) tại mỗi bước thời gian để xử lý chuỗi:

$$\boxed{h_t} = \boxed{f_W} \left(\boxed{x_t}, \boxed{h_{t-1}} \right)$$

trạng thái ô (cell state) hàm với trọng số W đầu vào (input) trạng thái cũ (old state)

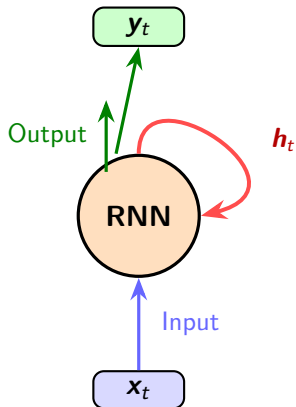
Lưu ý: Cùng một hàm f và cùng một bộ tham số W được sử dụng tại **mọi bước thời gian**.

RNN có một **trạng thái** (state) h_t được **cập nhật tại mỗi bước thời gian** khi xử lý chuỗi.

Recurrent Neural Network — Dạng cuộn (Rolled)

Ý nghĩa mũi tên vòng lại:

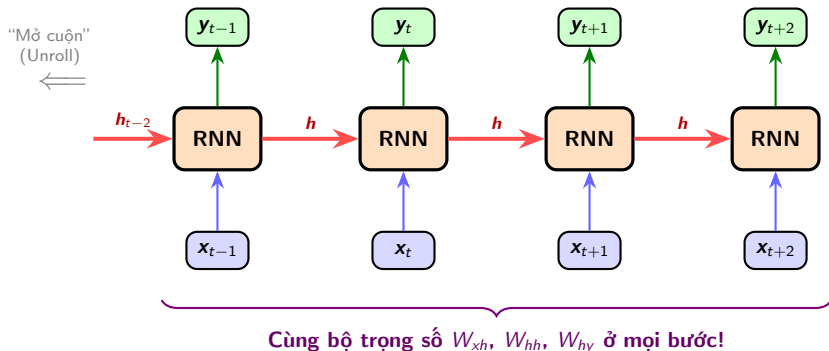
- Ở mỗi bước thời gian t , RNN nhận:
 - 1 Đầu vào hiện tại tại x_t
 - 2 Trạng thái ẩn trước đó h_{t-1}
- Tạo ra:
 - 1 Trạng thái ẩn mới h_t
 - 2 Đầu ra y_t (nếu cần)



Tương tự thực tế

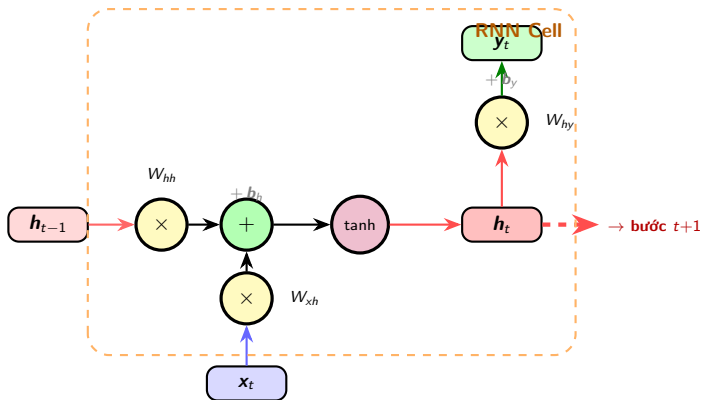
Như bạn đọc sách — mắt nhìn từ hiện tại (x_t), não vẫn nhớ nội dung đã đọc (h_{t-1}), kết hợp lại thành hiểu biết mới (h_t).

RNN dạng mở (Unrolled) qua thời gian



- Mỗi ô RNN **giống hệt nhau** — cùng tham số (parameter sharing)
- Dạng mở giúp ta thấy rõ **dòng chảy thông tin** qua thời gian
- Dạng cuộn và dạng mở là **hai cách vẽ cùng một mạng**

Bên trong một ô RNN (RNN Cell)



- h_{t-1} và x_t được **biến đổi tuyến tính** rồi **cộng lại**
- Hàm $\tanh$ nén kết quả vào khoảng $(-1, +1)$
- h_t vừa dùng để tính y_t , vừa **truyền sang bước tiếp theo**

Công thức toán cốt lõi của RNN

Bây giờ hãy viết lại chính xác những gì ta đã thấy trong sơ đồ:

Cập nhật trạng thái ẩn (Hidden State Update)

$$\mathbf{h}_t = \tanh(W_{hh} \mathbf{h}_{t-1} + W_{xh} \mathbf{x}_t + \mathbf{b}_h)$$

Tính đầu ra (Output)

$$\mathbf{y}_t = W_{hy} \mathbf{h}_t + \mathbf{b}_y$$

Ký hiệu:

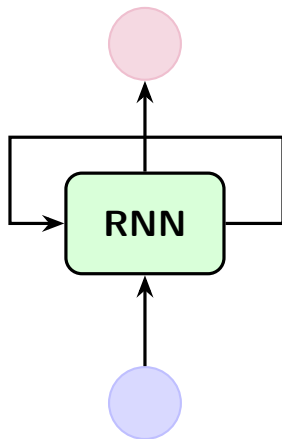
- $\mathbf{x}_t \in \mathbb{R}^d$: vector đầu vào tại bước t ; $\mathbf{h}_t \in \mathbb{R}^n$: trạng thái ẩn tại bước t
- $W_{xh} \in \mathbb{R}^{n \times d}$, $W_{hh} \in \mathbb{R}^{n \times n}$, $W_{hy} \in \mathbb{R}^{o \times n}$: ma trận trọng số
- $\mathbf{b}_h \in \mathbb{R}^n$, $\mathbf{b}_y \in \mathbb{R}^o$: bias vectors
- Tham số **chia sẻ** (shared) qua mọi bước thời gian

Mô hình hoá chuỗi: Các tiêu chí thiết kế

Để mô hình hoá chuỗi (sequences), ta cần:

- 1 Xử lý chuỗi có **độ dài thay đổi**
- 2 Theo dõi **phụ thuộc dài hạn** (long-term dependencies)
- 3 Duy trì thông tin về **thứ tự** các phần tử
- 4 **Chia sẻ tham số** xuyên suốt chuỗi

Mạng Neural Hồi quy (RNNs) đáp ứng tất cả các tiêu chí thiết kế này.



Bài toán mô hình chuỗi: Dự đoán từ tiếp theo

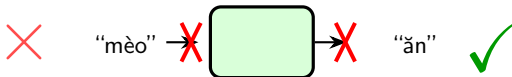
“Hà Nội là thủ đô của

cho trước các từ này

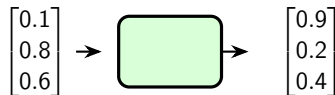
Việt Nam.”

dự đoán
từ tiếp theo

Biểu diễn ngôn ngữ cho mạng Neural



*Mạng neural không thể
xử lý trực tiếp chữ viết*



*Mạng neural cần đầu vào
dưới dạng số (vector)*

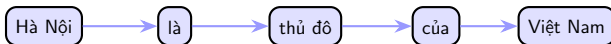
Từ câu văn đến vector: Làm sao để RNN “đọc” được?

Bước 1

“Hà Nội là thủ đô của Việt Nam”

Bước 2

Tách từ



Bước 3

Tra bảng từ vựng



Từ vựng:

0: tôi

1: anh

...

3: của

5: là

...

42: Hà Nội

...

108: thủ đô

...

Mỗi từ

→ 1 vector

kích thước

d

(ví dụ

$d = 3$)

Học được

từ dữ liệu!

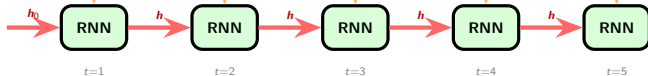
Bước 4

Embedding



Bước 5

Đưa vào RNN



Cách 1: One-hot Vector — Đơn giản nhưng hạn chế

Cách làm:

- Mỗi từ $\rightarrow$ 1 vector: **một vị trí = 1**, còn lại = 0
- Kích thước = kích thước bộ từ vựng

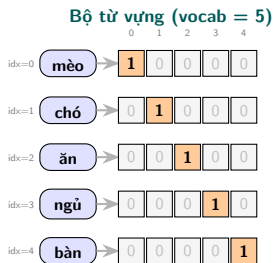
Ưu điểm:

- Đơn giản, **không cần huấn luyện**

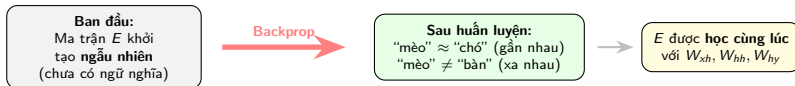
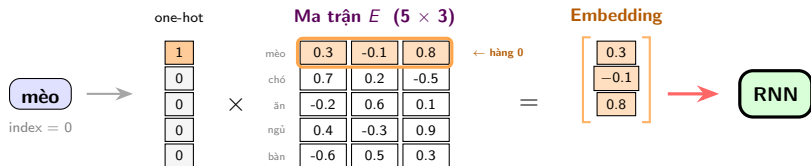
Nhược điểm:

- Vector rất **thưa** (sparse)
- Vocab 50.000 $\rightarrow$ vector 50.000 chiều!
- **Không có ngữ nghĩa**: “mèo”–“chó” cách xa y hệt “mèo”–“bàn”

Vấn đề: “Mèo” và “chó” liên quan nhau, nhưng one-hot không thể hiện được!



Cách 2: Learned Embedding — Mạng neural tự học biểu diễn



Tóm lại

Embedding **không có sẵn** — nó là một lớp trọng số được **học từ dữ liệu**. Tra bảng embedding = lấy hàng tương ứng trong ma trận E .

Ví dụ tính tay: Thiết lập bài toán

Bài toán: Cho chuỗi 3 từ trong bộ từ vựng {mèo, ăn, cá}, dự đoán từ tiếp theo.

Thiết lập (để đơn giản, bỏ bias):

- Vocab size = 3, hidden size = 2
- One-hot encoding: mèo = $[1, 0, 0]^T$, ăn = $[0, 1, 0]^T$, cá = $[0, 0, 1]^T$
- Trạng thái ẩn ban đầu: $\mathbf{h}_0 = [0, 0]^T$

Trọng số (chọn số nhỏ để dễ tính):

$$W_{xh} = \begin{bmatrix} 0.1 & 0.2 & 0.3 \\ 0.4 & 0.5 & 0.6 \end{bmatrix}, \quad W_{hh} = \begin{bmatrix} 0.1 & 0.2 \\ 0.3 & 0.4 \end{bmatrix}, \quad W_{hy} = \begin{bmatrix} 0.2 & 0.3 \\ 0.5 & 0.1 \\ 0.4 & 0.6 \end{bmatrix}$$

Chuỗi đầu vào: mèo $\rightarrow$ ăn $\rightarrow$ cá

Ta sẽ tính forward pass qua cả 3 bước thời gian.

Forward Pass — Bước $t = 1$: Đọc “mèo”

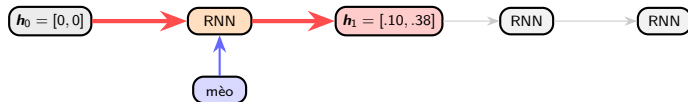
$$\mathbf{x}_1 = [1, 0, 0]^\top \text{ (one-hot của “mèo”)}, \quad \mathbf{h}_0 = [0, 0]^\top$$

Tính $\mathbf{h}_1$:

$$W_{xh} \mathbf{x}_1 = \begin{bmatrix} 0.1 & 0.2 & 0.3 \\ 0.4 & 0.5 & 0.6 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 0.1 \\ 0.4 \end{bmatrix}$$

$$W_{hh} \mathbf{h}_0 = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \quad (\text{vì } \mathbf{h}_0 = 0)$$

$$\mathbf{h}_1 = \tanh \left(\begin{bmatrix} 0.1 \\ 0.4 \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \end{bmatrix} \right) = \tanh \begin{bmatrix} 0.1 \\ 0.4 \end{bmatrix} = \begin{bmatrix} 0.0997 \\ 0.3799 \end{bmatrix}$$



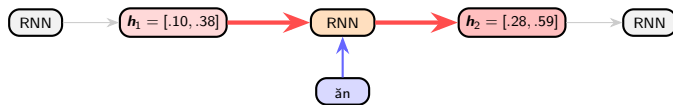
Forward Pass — Bước $t = 2$: Đọc “ăn”

$$\mathbf{x}_2 = [0, 1, 0]^\top, \quad \mathbf{h}_1 = [0.0997, 0.3799]^\top$$

Tính $\mathbf{h}_2$:

$$W_{xh} \mathbf{x}_2 = \begin{bmatrix} 0.2 \\ 0.5 \end{bmatrix}, \quad W_{hh} \mathbf{h}_1 = \begin{bmatrix} 0.1 \cdot 0.10 + 0.2 \cdot 0.38 \\ 0.3 \cdot 0.10 + 0.4 \cdot 0.38 \end{bmatrix} = \begin{bmatrix} 0.086 \\ 0.182 \end{bmatrix}$$

$$\mathbf{h}_2 = \tanh \begin{bmatrix} 0.286 \\ 0.682 \end{bmatrix} = \begin{bmatrix} 0.2793 \\ 0.5928 \end{bmatrix}$$



$\mathbf{h}_2$ giờ “nhớ” cả mèo (qua $\mathbf{h}_1$) lẫn ăn (qua $\mathbf{x}_2$).

Forward Pass — Bước $t = 3$: Đọc “cá” & Dự đoán

$$\mathbf{x}_3 = [0, 0, 1]^\top, \quad \mathbf{h}_2 = [0.2793, 0.5928]^\top$$

Tính $\mathbf{h}_3$:

$$W_{xh} \mathbf{x}_3 = \begin{bmatrix} 0.3 \\ 0.6 \end{bmatrix}, \quad W_{hh} \mathbf{h}_2 = \begin{bmatrix} 0.146 \\ 0.321 \end{bmatrix}$$

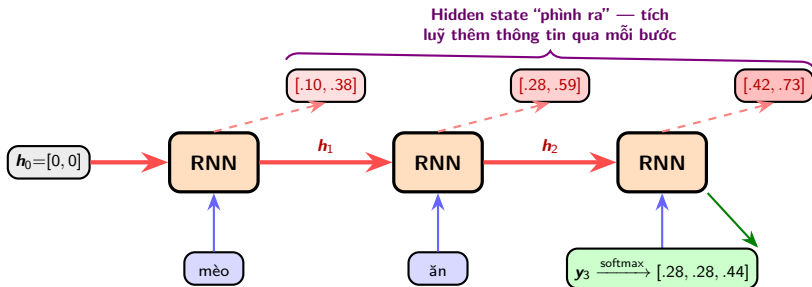
$$\mathbf{h}_3 = \tanh \begin{bmatrix} 0.446 \\ 0.921 \end{bmatrix} = \begin{bmatrix} 0.4188 \\ 0.7259 \end{bmatrix}$$

Tính đầu ra $\mathbf{y}_3$ (logits):

$$\mathbf{y}_3 = W_{hy} \mathbf{h}_3 = \begin{bmatrix} 0.2 \cdot 0.42 + 0.3 \cdot 0.73 \\ 0.5 \cdot 0.42 + 0.1 \cdot 0.73 \\ 0.4 \cdot 0.42 + 0.6 \cdot 0.73 \end{bmatrix} \approx \begin{bmatrix} 0.302 \\ 0.283 \\ 0.604 \end{bmatrix} \xrightarrow{\text{softmax}} \begin{bmatrix} 0.284 \\ 0.279 \\ 0.437 \end{bmatrix}$$

⇒ Xác suất cao nhất: từ thứ 3 = cá (chưa huấn luyện nên kết quả chưa tốt!)

Toàn cảnh Forward Pass



Quan sát:

- Giá trị trong h_t **tăng dần** khi tích lũy thêm thông tin
- h_3 chứa dấu vết của **cả 3 từ** đã đọc
- Sau huấn luyện (backprop), trọng số sẽ được điều chỉnh để dự đoán tốt hơn

Chuỗi thời gian vào RNN: Đã là số — không cần Embedding!

Văn bản:

“mèo” $\xrightarrow{\text{embedding}}$ $[0.3, -0.1, 0.8]$

cần chuyển đổi

Giá cổ phiếu:

\$150.2 $\rightarrow$ dùng trực tiếp!

đã là số sẵn

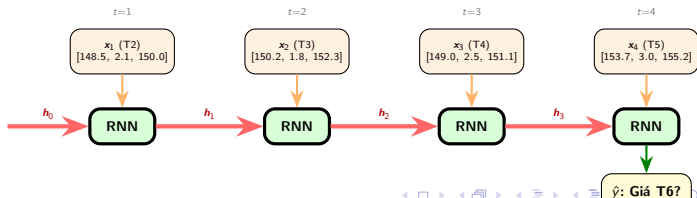
Bảng dữ liệu gốc:

Ngày	Giá đóng cửa	Khối lượng	Giá cao nhất
T2	148.5	2.1M	150.0
T3	150.2	1.8M	152.3
T4	149.0	2.5M	151.1
T5	153.7	3.0M	155.2

Mỗi hàng (ngày)
= 1 vector x_t
kích thước $d = 3$

$x_1 = [148.5, 2.1, 150.0]$

Đưa vào RNN theo thứ tự thời gian:

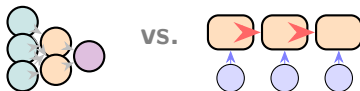


Tổng kết: Feedforward Network vs. RNN

Tiêu chí	Feedforward NN	Recurrent NN
Kiến trúc	Một chiều, không vòng lặp	Có vòng lặp (self-loop)
Bộ nhớ	Không	Có (hidden state h_t)
Đầu vào	Kích thước cố định	Chuỗi có độ dài bất kỳ
Chia sẻ tham số	Không (mỗi lớp riêng)	Có (cùng W mọi bước)
Phù hợp với	Ảnh, dữ liệu bảng	Văn bản, chuỗi thời gian
Huấn luyện	Backpropagation	Backprop Through Time

Feedforward

RNN (unrolled)



Hạn chế của RNN cơ bản & Hướng đi tiếp

RNN cơ bản (Vanilla RNN) vẫn có hạn chế:

- **Vanishing/Exploding Gradient:** gradient “biến mất” hoặc “bùng nổ” khi chuỗi dài
 - ▶ h_{100} rất khó nhớ thông tin từ h_1
- **Khó xử lý phụ thuộc xa** (long-range dependency)
 - ▶ “Con mèo, con mà tôi nuôi từ nhỏ, _____ rất đáng yêu.”
 - ▶ Cần nhớ “con mèo” qua rất nhiều từ ở giữa

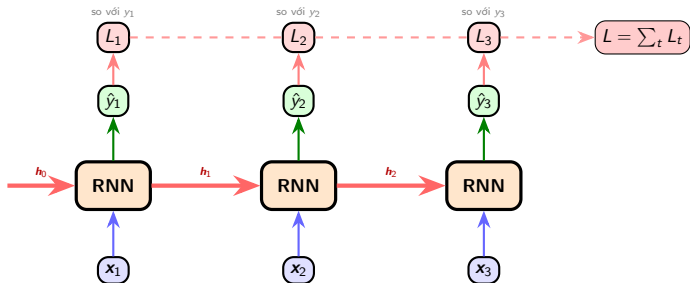
Giải pháp (sẽ học ở các bài sau):

- ① **LSTM** (Long Short-Term Memory) — thêm “cổng” để kiểm soát dòng thông tin
- ② **GRU** (Gated Recurrent Unit) — phiên bản đơn giản hơn LSTM
- ③ **Transformer & Attention** — cách tiếp cận hoàn toàn mới, không cần tuần tự

Ghi nhớ

RNN là **nền tảng tư duy** cho mọi mô hình xử lý chuỗi — hiểu RNN giúp bạn hiểu sâu hơn LSTM, GRU, và Transformer.

Nhắc lại: Forward Pass trong RNN

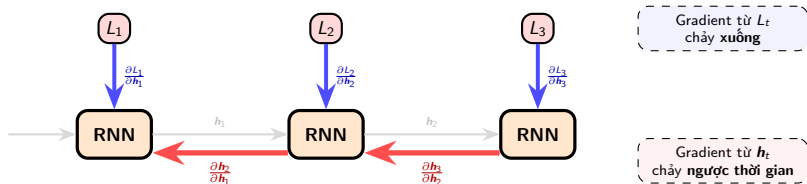


Công thức forward:

$$\mathbf{h}_t = \tanh(W_{hh} \mathbf{h}_{t-1} + W_{xh} \mathbf{x}_t), \quad \hat{y}_t = W_{hy} \mathbf{h}_t, \quad L = \sum_{t=1}^T L_t(\hat{y}_t, y_t)$$

Câu hỏi: Làm sao tính $\frac{\partial L}{\partial W}$ khi W được dùng ở **mọi** bước thời gian?

Ý tưởng BPTT: Gradient “chảy ngược” qua thời gian



BPTT = Backpropagation Through Time

- “Mở cuộn” RNN thành mạng feedforward dài T bước
- Áp dụng **chain rule** xuyên suốt chuỗi thời gian
- Gradient phải “đi ngược” từ $t = T$ về $t = 1$ qua mọi hidden state

Chain Rule xuyên thời gian

Mục tiêu: Tính $\frac{\partial L}{\partial W_{hh}}$ (tương tự cho W_{xh} , W_{hy})

Vì W_{hh} xuất hiện ở **mọi** bước, ta phải cộng đóng góp từ **tất cả** các bước:

$$\frac{\partial L}{\partial W_{hh}} = \sum_{t=1}^T \frac{\partial L_t}{\partial W_{hh}}$$

Chain Rule xuyên thời gian

Mục tiêu: Tính $\frac{\partial L}{\partial W_{hh}}$ (tương tự cho W_{xh} , W_{hy})

Vì W_{hh} xuất hiện ở **mọi** bước, ta phải cộng đóng góp từ **tất cả** các bước:

$$\frac{\partial L}{\partial W_{hh}} = \sum_{t=1}^T \frac{\partial L_t}{\partial W_{hh}}$$

Tại mỗi bước t , gradient phải “chảy ngược” qua các hidden state:

$$\frac{\partial L_t}{\partial W_{hh}} = \sum_{k=1}^t \frac{\partial L_t}{\partial \hat{y}_t} \cdot \frac{\partial \hat{y}_t}{\partial \mathbf{h}_t} \cdot \underbrace{\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_k}}_{\text{chain qua thời gian}} \cdot \frac{\partial \mathbf{h}_k}{\partial W_{hh}}$$

Chain Rule xuyên thời gian

Mục tiêu: Tính $\frac{\partial L}{\partial W_{hh}}$ (tương tự cho W_{xh} , W_{hy})

Vì W_{hh} xuất hiện ở **mọi** bước, ta phải cộng đóng góp từ **tất cả** các bước:

$$\frac{\partial L}{\partial W_{hh}} = \sum_{t=1}^T \frac{\partial L_t}{\partial W_{hh}}$$

Tại mỗi bước t , gradient phải “chảy ngược” qua các hidden state:

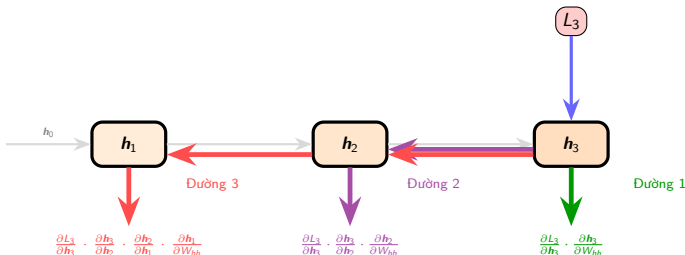
$$\frac{\partial L_t}{\partial W_{hh}} = \sum_{k=1}^t \frac{\partial L_t}{\partial \hat{y}_t} \cdot \frac{\partial \hat{y}_t}{\partial \mathbf{h}_t} \cdot \underbrace{\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_k}}_{\text{chain qua thời gian}} \cdot \frac{\partial \mathbf{h}_k}{\partial W_{hh}}$$

Phần quan trọng nhất — gradient truyền qua nhiều bước:

$$\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_k} = \prod_{i=k+1}^t \frac{\partial \mathbf{h}_i}{\partial \mathbf{h}_{i-1}} = \prod_{i=k+1}^t \text{diag}(\tanh'(\cdot)) \cdot W_{hh}$$

⇒ Tích của $(t - k)$ ma trận! Đây là nguồn gốc vấn đề vanishing/exploding gradient.

Trực quan: Gradient từ L_3 chảy về W_{hh} qua bao nhiêu đường?

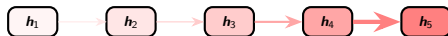


$$\frac{\partial L_3}{\partial W_{hh}} = \underbrace{\text{Đường 1}}_{\text{tại } t=3} + \underbrace{\text{Đường 2}}_{\text{qua } h_2} + \underbrace{\text{Đường 3}}_{\text{qua } h_1} = \sum_{k=1}^3 \frac{\partial L_3}{\partial h_3} \cdot \frac{\partial h_3}{\partial h_k} \cdot \frac{\partial h_k}{\partial W_{hh}}$$

Đường càng dài $\Rightarrow$ tích càng nhiều ma trận $\Rightarrow$ gradient dễ **biến mất** hoặc **bùng nổ**.

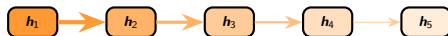
Vấn đề: Vanishing & Exploding Gradient

Vanishing:



Gradient nhỏ dần
→ không học được
phụ thuộc xa

Exploding:



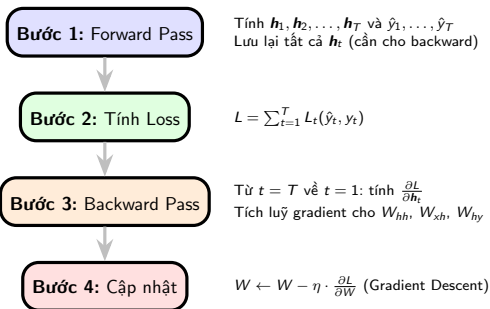
Gradient lớn dần
→ trọng số “nhảy”
huấn luyện bất ổn

Nguyên nhân toán học:

$$\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_k} = \prod_{i=k+1}^t \underbrace{\text{diag}(\tanh')}_{\leq 1} \cdot W_{hh}$$

- Nếu $\|W_{hh}\| < 1$: tích nhiều số $< 1 \Rightarrow$ **vanishing** (gradient $\rightarrow 0$)
- Nếu $\|W_{hh}\| > 1$: tích nhiều số $> 1 \Rightarrow$ **exploding** (gradient $\rightarrow \infty$)
- $\tanh'(x) \in (0, 1]$ luôn $\leq 1 \Rightarrow$ càng dễ vanishing hơn

Thuật toán BPTT — Tổng quan từng bước



Chú ý quan trọng

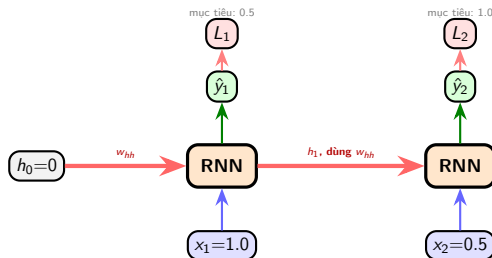
Bước 3 là phần **khác biệt** so với backprop thường: gradient tại mỗi $\mathbf{h}_t$ nhận đóng góp từ cả L_t (phía trên) lẫn $\mathbf{h}_{t+1}$ (phía phải).

Ví dụ tính tay BPTT — Thiết lập

Bài toán đơn giản: RNN với 1 input, 1 hidden unit, 1 output, chuỗi 2 bước.

Cho (bỏ bias, tất cả là số vô hướng):

- Trọng số: $w_{xh} = 0.5$, $w_{hh} = 0.8$, $w_{hy} = 0.7$
- $h_0 = 0$, đầu vào: $x_1 = 1.0$, $x_2 = 0.5$, mục tiêu: $y_1 = 0.5$, $y_2 = 1.0$
- Loss: $L_t = \frac{1}{2}(\hat{y}_t - y_t)^2$, hidden: tanh, output: linear



Ví dụ tính tay — Bước 1: Forward Pass

Bước $t = 1$: $x_1 = 1.0$, $h_0 = 0$

$$h_1 = \tanh(w_{hh} \cdot h_0 + w_{xh} \cdot x_1) = \tanh(0.8 \cdot 0 + 0.5 \cdot 1.0) = \tanh(0.5) = 0.4621$$

$$\hat{y}_1 = w_{hy} \cdot h_1 = 0.7 \times 0.4621 = 0.3235$$

$$L_1 = \frac{1}{2}(0.3235 - 0.5)^2 = \frac{1}{2}(-0.1765)^2 = 0.0156$$

Bước $t = 2$: $x_2 = 0.5$, $h_1 = 0.4621$

$$\begin{aligned} h_2 &= \tanh(w_{hh} \cdot h_1 + w_{xh} \cdot x_2) = \tanh(0.8 \times 0.4621 + 0.5 \times 0.5) \\ &= \tanh(0.3697 + 0.25) = \tanh(0.6197) = 0.5511 \end{aligned}$$

$$\hat{y}_2 = w_{hy} \cdot h_2 = 0.7 \times 0.5511 = 0.3858$$

$$L_2 = \frac{1}{2}(0.3858 - 1.0)^2 = \frac{1}{2}(-0.6142)^2 = 0.1886$$

$$L = L_1 + L_2 = 0.0156 + 0.1886 = 0.2042$$

Ví dụ tính tay — Bước 2: Backward tại $t = 2$

Bắt đầu từ bước cuối cùng, tính ngược lại:

Gradient của loss tại $t = 2$:

$$\frac{\partial L_2}{\partial \hat{y}_2} = \hat{y}_2 - y_2 = 0.3858 - 1.0 = -0.6142$$

Gradient qua w_{hy} :

$$\frac{\partial L_2}{\partial w_{hy}} = \frac{\partial L_2}{\partial \hat{y}_2} \cdot h_2 = (-0.6142) \times 0.5511 = -0.3385$$

Gradient chảy về h_2 :

$$\frac{\partial L_2}{\partial h_2} = \frac{\partial L_2}{\partial \hat{y}_2} \cdot w_{hy} = (-0.6142) \times 0.7 = -0.4300$$

Gradient qua tanh (cần $\tanh'$):

$$\tanh'(0.6197) = 1 - \tanh^2(0.6197) = 1 - 0.5511^2 = 1 - 0.3037 = 0.6963$$

Ví dụ tính tay — Backward tại $t = 2$ (tiếp): Gradient cho

w_{hh}

Gọi $z_2 = w_{hh} \cdot h_1 + w_{xh} \cdot x_2 = 0.6197$ (giá trị trước tanh)

Gradient w_{hh} tại bước $t = 2$ (đóng góp trực tiếp):

$$\begin{aligned}\left. \frac{\partial L_2}{\partial w_{hh}} \right|_{\text{tại } t=2} &= \frac{\partial L_2}{\partial h_2} \cdot \tanh'(z_2) \cdot \frac{\partial z_2}{\partial w_{hh}} \\ &= (-0.4300) \times 0.6963 \times h_1 \\ &= (-0.4300) \times 0.6963 \times 0.4621 = -0.1383\end{aligned}$$

Ví dụ tính tay — Backward tại $t = 2$ (tiếp): Gradient cho

w_{hh}

Gọi $z_2 = w_{hh} \cdot h_1 + w_{xh} \cdot x_2 = 0.6197$ (giá trị trước tanh)

Gradient w_{hh} tại bước $t = 2$ (đóng góp trực tiếp):

$$\begin{aligned}\left. \frac{\partial L_2}{\partial w_{hh}} \right|_{\text{tại } t=2} &= \frac{\partial L_2}{\partial h_2} \cdot \tanh'(z_2) \cdot \frac{\partial z_2}{\partial w_{hh}} \\ &= (-0.4300) \times 0.6963 \times h_1 \\ &= (-0.4300) \times 0.6963 \times 0.4621 = -0.1383\end{aligned}$$

Gradient truyền ngược về h_1 (để tiếp tục chuỗi):

$$\begin{aligned}\frac{\partial L_2}{\partial h_1} &= \frac{\partial L_2}{\partial h_2} \cdot \tanh'(z_2) \cdot w_{hh} \\ &= (-0.4300) \times 0.6963 \times 0.8 = -0.2395\end{aligned}$$

⇒ Gradient “chảy ngược” từ h_2 về h_1 bị nhân với $\tanh'(\cdot) \times w_{hh}$. Đây chính là cơ chế vanishing!

Ví dụ tính tay — Bước 3: Backward tại $t = 1$

Tại $t = 1$, gradient cho h_1 nhận **hai nguồn**:

- ❶ Từ L_1 (trực tiếp):

$$\frac{\partial L_1}{\partial h_1} = (\hat{y}_1 - y_1) \cdot w_{hy} = (-0.1765) \times 0.7 = -0.1236$$

- ❷ Từ L_2 (qua h_2 , đã tính ở slide trước): $\frac{\partial L_2}{\partial h_1} = -0.2395$

Tổng gradient tại h_1 :

$$\frac{\partial L}{\partial h_1} = \frac{\partial L_1}{\partial h_1} + \frac{\partial L_2}{\partial h_1} = -0.1236 + (-0.2395) = -0.3631$$

Gradient w_{hh} tại bước $t = 1$: ($z_1 = 0.5$,
 $\tanh'(0.5) = 1 - 0.4621^2 = 0.7864$)

$$\left. \frac{\partial L}{\partial w_{hh}} \right|_{\text{tại } t=1} = \frac{\partial L}{\partial h_1} \cdot \tanh'(z_1) \cdot h_0 = (-0.3631) \times 0.7864 \times 0 = 0$$

(Bằng 0 vì $h_0 = 0$. Trong thực tế $h_0 \neq 0$ thì sẽ có đóng góp.)

Ví dụ tính tay — Tổng hợp kết quả

Tổng gradient cho từng trọng số:

Trọng số	Đóng góp từ $t=1$	Đóng góp từ $t=2$	Tổng
w_{hy}	-0.0816	-0.3385	-0.4201
w_{hh}	0 (vì $h_0 = 0$)	-0.1383	-0.1383
w_{xh}	-0.2856	-0.1497	-0.4353

Cập nhật trọng số (learning rate $\eta = 0.1$):

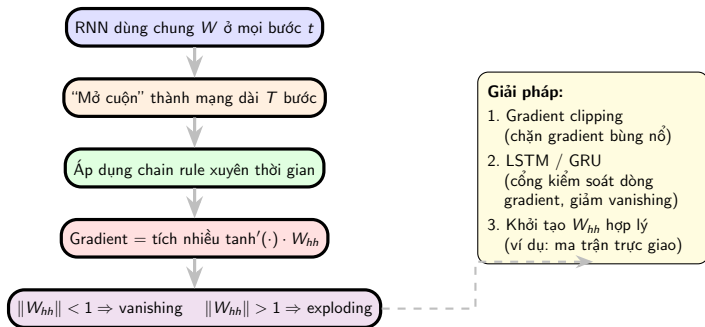
$$w_{hy} \leftarrow 0.7 - 0.1 \times (-0.4201) = 0.7420$$

$$w_{hh} \leftarrow 0.8 - 0.1 \times (-0.1383) = 0.8138$$

$$w_{xh} \leftarrow 0.5 - 0.1 \times (-0.4353) = 0.5435$$

Gradient âm $\Rightarrow$ trọng số **tăng** $\Rightarrow$ output lớn hơn $\Rightarrow$ gần mục tiêu hơn. Đúng!

Tóm tắt: BPTT trong RNN



Ghi nhớ

BPTT = Backpropagation + Time. Hiểu BPTT giúp bạn hiểu **tại sao** cần LSTM/GRU và **tại sao** Transformer ra đời.

Đặt vấn đề: RNN có thật sự “nhớ” được lâu?

Thử thách: Đoán từ cuối trong hai câu sau.

“Bầu trời có màu _____.”

← Dễ! (xanh)

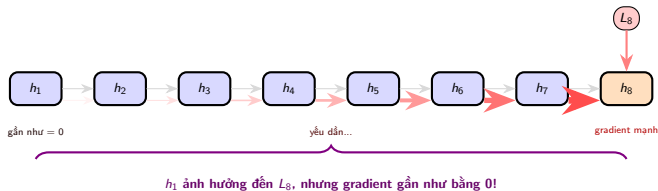
“Tôi lớn lên ở **Pháp**. Sau nhiều năm học tập và làm việc ở nhiều nơi trên thế giới, tôi nói thành thạo tiếng _____.”

← Cần nhớ “Pháp” qua rất nhiều từ ở giữa mới đoán được “Pháp”!

Vấn đề cốt lõi

RNN lý thuyết có thể nhớ mọi thứ qua hidden state. Nhưng thực tế, khi huấn luyện bằng BPTT, **gradient biến mất** trước khi truyền được đến các bước xa $\Rightarrow$ RNN **không học được** phụ thuộc xa.

Gradient phải “đi qua” bao nhiêu bước?



Nhắc lại công thức:

$$\frac{\partial h_t}{\partial h_k} = \prod_{i=k+1}^t \underbrace{\tanh'(\cdot)}_{\leq 1} \cdot \underbrace{w_{hh}}_{\text{cùng giá trị}}$$

Nếu $|\tanh'(\cdot) \cdot w_{hh}| < 1 \Rightarrow$ nhân $(t - k)$ lần $\Rightarrow$ kết quả $\rightarrow 0$ rất nhanh!

Ví dụ số: Gradient biến mất nhanh như thế nào?

Giả sử tại mỗi bước: $\tanh'(\cdot) = 0.8$ và $w_{hh} = 0.5$

⇒ Hệ số nhân mỗi bước: $0.8 \times 0.5 = 0.4$

Khoảng cách	$t-k=1$	2	5	10	20	50
$\frac{\partial h_t}{\partial h_k}$	0.4	0.16	0.01	10^{-4}	10^{-8}	10^{-20}

Gradient → 0 theo cấp số nhân!

Hậu quả:

- Trọng số ở bước xa **không được cập nhật**
- RNN chỉ “nhớ” được vài bước gần
- Phụ thuộc dài hạn → không học được

Công thức tổng quát:

$$0.4^{t-k} \xrightarrow{t-k \rightarrow \infty} 0$$

Bất kỳ số nào < 1 , nhân nhiều lần đều $\rightarrow 0$.

Mặt Khác: Gradient bùng nổ (Exploding)

Giả sử $\tanh'(\cdot) = 0.9$ và $w_{hh} = 1.5$

⇒ Hệ số nhân mỗi bước: $0.9 \times 1.5 = 1.35$

Khoảng cách	$t-k=1$	2	5	10	20	50
$\frac{\partial h_t}{\partial h_k}$	1.35	1.82	4.48	20.1	404	1.6×10^6

Gradient $\rightarrow \infty$ theo cấp số nhân!

Hậu quả:

- Trọng số bị cập nhật **quá lớn**
- Loss “nhảy” lung tung, không hội tụ
- Giá trị trở thành NaN

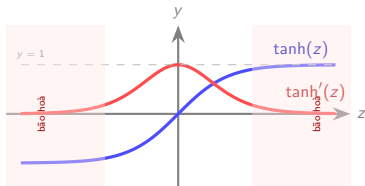
Dấu hiệu nhận biết:

- Loss đột ngột tăng vọt
- Trọng số trở thành NaN/Inf
- Huấn luyện “chết” đột ngột

Tại sao tanh làm trầm trọng thêm Vanishing?

Quan sát:

- $\tanh'(z)$ đạt cực đại $= 1$ tại $z = 0$
- $|z|$ lớn $\Rightarrow \tanh'(z) \rightarrow 0$ (vùng **bão hoà**)
- $\tanh'(z) \in (0, 1]$ **luôn** ≤ 1



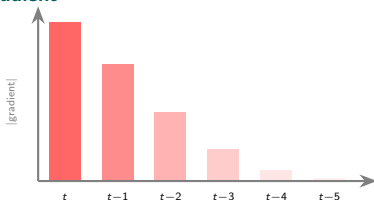
Hệ quả cho gradient:

$$\prod_{i=k+1}^t \underbrace{\tanh'(\cdot)}_{\leq 1} \cdot w_{hh}$$

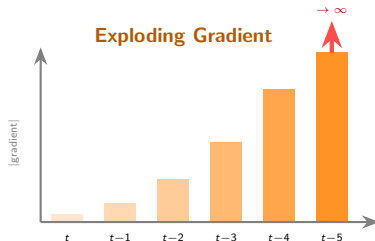
- Mỗi bước nhân thêm 1 số ≤ 1
- Ngay cả khi $w_{hh} \approx 1$, $\tanh'$ vẫn kéo gradient xuống
- Vanishing xảy ra **dễ hơn** exploding

So sánh: Vanishing vs. Exploding Gradient

Vanishing Gradient



Exploding Gradient



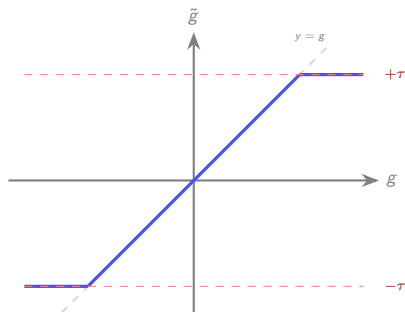
	Vanishing	Exploding
Điều kiện	$ \tanh' \cdot w_{hh} < 1$	$ \tanh' \cdot w_{hh} > 1$
Triệu chứng	Loss giảm rồi “đứng yên”	Loss nhảy lên NaN
Phổ biến?	Rất phổ biến	Ít hơn, dễ phát hiện
Khó sửa?	Khó (cần thay kiến trúc)	Dễ hơn (gradient clipping)

Giải pháp 1: Gradient Clipping (chặn gradient bùng nổ)

Ý tưởng: Nếu gradient quá lớn, thu nhỏ nó lại.

Cách làm (norm clipping):

$$\tilde{g} = \begin{cases} g & \text{nếu } \|g\| \leq \tau \\ \frac{\tau}{\|g\|} \cdot g & \text{nếu } \|g\| > \tau \end{cases}$$

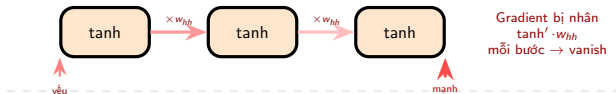


Gradient sau khi clip

- τ là ngưỡng (threshold), thường = 1 hoặc 5
- Giữ nguyên **hướng** gradient, chỉ giảm **độ lớn**
- Chỉ giải quyết **exploding**, **không** giải quyết **vanishing**!

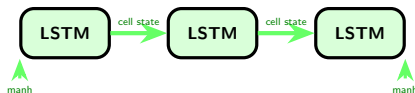
Giải pháp 2: Thay đổi kiến trúc $\Rightarrow$ LSTM & GRU

Vanilla RNN



Gradient bị nhân $\tanh' \cdot w_{hh}$ mỗi bước $\rightarrow$ vanish

LSTM / GRU

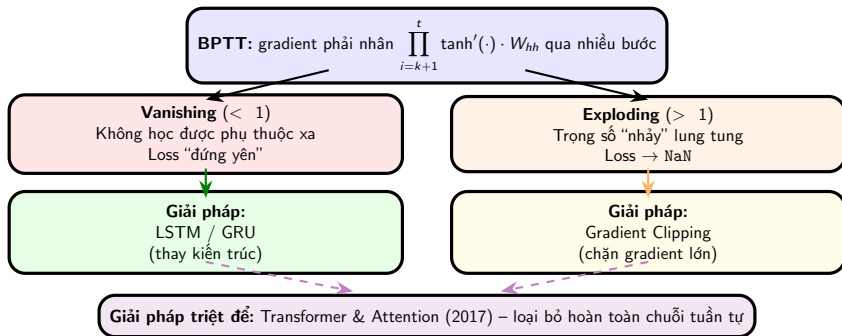


Gradient chảy qua “đường cao tốc” (cell state) gần như không bị suy giảm

Ý tưởng chính của LSTM/GRU

Thêm **cell state** — một “đường cao tốc” cho gradient chảy qua nhiều bước mà **không bị nhân** với $\tanh' \cdot w_{hh}$ mỗi bước. Các “cổng” (gates) quyết định thông tin nào được thêm/xoá.

Tổng kết: Vanishing & Exploding Gradient trong RNN

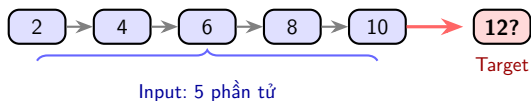


Ghi nhớ

Vanishing gradient là lý do chính khiến RNN cơ bản “thất bại” với chuỗi dài, và là **động lực** ra đời LSTM (1997), GRU (2014), rồi Transformer (2017).

Bài toán: Dự đoán phần tử tiếp theo trong chuỗi

Mục tiêu: Cho chuỗi 5 số (cấp số cộng), dự đoán số thứ 6.

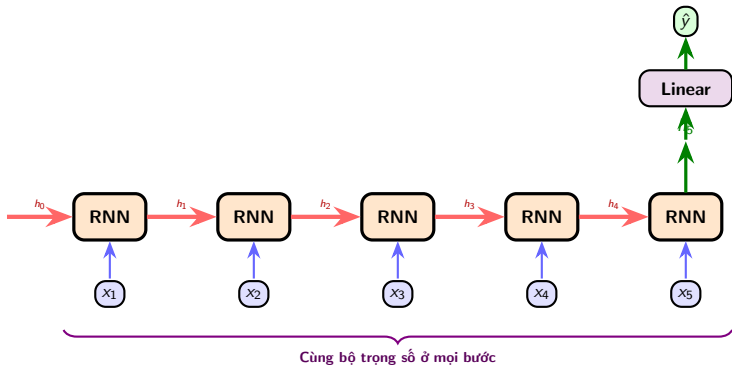


Dữ liệu: Tự sinh các chuỗi cấp số cộng ngẫu nhiên

- $[1, 3, 5, 7, 9] \rightarrow 11$ (công sai $d = 2$)
- $[5, 8, 11, 14, 17] \rightarrow 20$ (công sai $d = 3$)

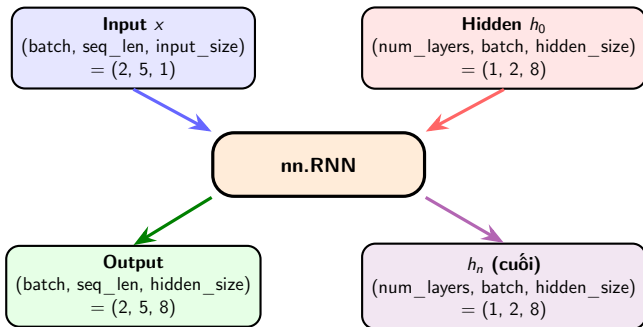
Công cụ: `torch.nn.RNN` — RNN cơ bản nhất của PyTorch

Kiến trúc mô hình



Luồng xử lý: Input (5 số) $\xrightarrow{\text{RNN}}$ Hidden state tại mỗi bước $\xrightarrow{\text{lấy } h_5}$ Lớp Linear $\rightarrow$ Dự đoán 1 số

Các shape quan trọng trong torch.nn.RNN



- `batch_first=True`: chiều đầu tiên là batch (thuận tiện hơn)
- **Output**: hidden state tại **mọi** bước → dùng để phân tích
- h_n : hidden state bước **cuối cùng** → thường dùng để dự đoán
- h_n chính là `output[:, -1, :]` (cột cuối của output)

Code: Tự sinh dữ liệu cấp số cộng

```
import torch, torch.nn as nn, numpy as np

def tao_du_lieu(so_mau=100, seq_len=5):
    X, y = [], []
    for _ in range(so_mau):
        a = np.random.randint(1, 10)    # số đầu
        d = np.random.randint(1, 5)    # công sai
        chuoi = [a + i*d for i in range(seq_len + 1)]
        X.append(chuoi[:-1])    # 5 phần tử = input
        y.append(chuoi[-1])    # phần tử 6 = target
    return torch.FloatTensor(X)/50, torch.FloatTensor(y)/50

X_train, y_train = tao_du_lieu(200)
# X_train.shape = [200, 5]    y_train.shape = [200]
# VD: [2,4,6,8,10]/50 -> target 12/50
```

Giải thích:

- Mỗi chuỗi là cấp số cộng ngẫu nhiên (số đầu 1–9, công sai 1–4)
- Chia 50 để normalize về khoảng [0, 1] (giúp huấn luyện ổn định)
- Input: 5 phần tử đầu, Target: phần tử thứ 6

Code: Xây dựng mô hình RNN

```
class RNN_DuDoan(nn.Module):
    def __init__(self, input_size=1, hidden_size=8):
        super().__init__()
        self.hidden_size = hidden_size
        self.rnn = nn.RNN(input_size, hidden_size,
                           num_layers=1, batch_first=True)
        self.fc = nn.Linear(hidden_size, 1)

    def forward(self, x):  # x: (batch, seq_len, 1)
        h0 = torch.zeros(1, x.size(0), self.hidden_size)
        output, hn = self.rnn(x, h0)
        # output: (batch, seq_len, hidden_size) - moi buoc
        # hn:      (1, batch, hidden_size) - buoc cuoi
        last_h = output[:, -1, :]  # lay h buoc cuoi
        return self.fc(last_h).squeeze(-1)  # -> (batch,)
```

model = RNN_DuDoan(input_size=1, hidden_size=8)

Tong: RNN(1->8) + Linear(8->1) = 97 tham so

Điểm quan trọng:

- batch_first=True: input có shape (batch, seq_len, features)
- output[:, -1, :] lấy hidden state bước **cuối** để dự đoán
- h0 = zeros: hidden state ban đầu = vector không

Code: Forward Pass chi tiết — Xem tensor từng bước

```
mau = X_train[:2].unsqueeze(-1) # (2,5)->(2,5,1)
h0 = torch.zeros(1, 2, 8)      # (layers, batch, hidden)

with torch.no_grad():
    output, hn = model.rnn(mau, h0)

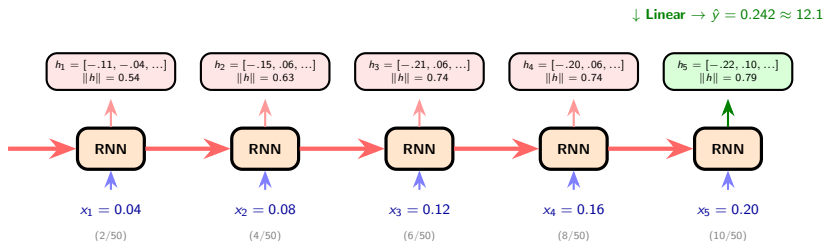
print(f"output: {output.shape}") # (2, 5, 8)
print(f"hn:      {hn.shape}")    # (1, 2, 8)

# Hidden state tai tung buoc (mau 1, 4 chieu dau):
# t=1: h=[ 0.280, -0.042, -0.293, -0.286]
# t=2: h=[ 0.364,  0.206, -0.492, -0.448]
# t=3: h=[ 0.412,  0.315, -0.596, -0.384]
# t=4: h=[ 0.414,  0.333, -0.652, -0.382]
# t=5: h=[ 0.400,  0.334, -0.677, -0.359]

# Kiem tra: hn == output buoc cuoi
print(torch.allclose(hn[0], output[:,-1,:])) # True
```

Quan sát: Hidden state thay đổi dần qua mỗi bước → tích lũy thông tin.

Minh họa: Hidden state thay đổi qua từng bước



Quan sát:

- Hidden state **thay đổi** mỗi bước khi nhận input mới
- Norm $\|h\|$ tăng dần → tích lũy thêm thông tin
- h_5 chứa “tóm tắt” toàn bộ chuỗi [2, 4, 6, 8, 10]
- Lớp Linear dùng h_5 để dự đoán $\hat{y} \approx 12$

Code: Huấn luyện mô hình

```
X_rnn = X_train.unsqueeze(-1)  # (200,5) -> (200,5,1)
criterion = nn.MSELoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.01)

for epoch in range(300):
    y_pred = model(X_rnn)
    loss = criterion(y_pred, y_train)
    optimizer.zero_grad()
    loss.backward()           # BPTT!
    nn.utils.clip_grad_norm_(model.parameters(), 1.0)
    optimizer.step()
    if (epoch+1) % 100 == 0:
        print(f"Epoch {epoch+1}: Loss={loss.item():.6f}")
# Epoch 100: Loss=0.001535
# Epoch 200: Loss=0.000011
# Epoch 300: Loss=0.000009
```

Điểm quan trọng:

- `loss.backward()` thực hiện **BPTT** (gradient chảy ngược qua thời gian)
- `clip_grad_norm_`: chặn gradient bùng nổ (exploding gradient)
- Loss giảm từ ~ 0.1 xuống $\sim 10^{-5}$ sau 300 epochs

Code: Kiểm tra kết quả trên dữ liệu mới

```
chuoi_moi = [[2,4,6,8,10], [5,8,11,14,17],  
             [1,3,5,7,9],  [10,13,16,19,22]]  
targets    = [12, 20, 11, 25]  
  
inp = torch.FloatTensor(chuoi_moi)/50  
inp = inp.unsqueeze(-1)           # (4, 5, 1)  
  
model.eval()  
with torch.no_grad():  
    pred = model(inp) * 50        # un-normalize  
  
for i in range(4):  
    print(f"{chuoi_moi[i]} -> {pred[i]:.1f} "  
          f"(Target: {targets[i]})")  
# [2,4,6,8,10]      -> 12.1 (Target: 12)  
# [5,8,11,14,17]    -> 20.1 (Target: 20)  
# [1,3,5,7,9]       -> 11.1 (Target: 11)  
# [10,13,16,19,22] -> 25.0 (Target: 25)
```

Sai số < 0.2 trên chuỗi **chưa thấy bao giờ** $\Rightarrow$ RNN đã học được quy luật cấp số cộng!

RNN học gì từ chuỗi?

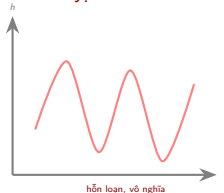
Trước huấn luyện:

- W_{xh} , W_{hh} , W_{hy} : **ngẫu nhiên**
- Hidden state: vô nghĩa
- $[2, 4, 6, 8, 10] \rightarrow -11.2$ ✗

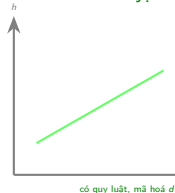
Sau huấn luyện:

- Trọng số điều chỉnh bởi BPTT
- Hidden state mã hoá **quy luật chuỗi**
- $[2, 4, 6, 8, 10] \rightarrow 12.1$ ✓

Trước huấn luyện



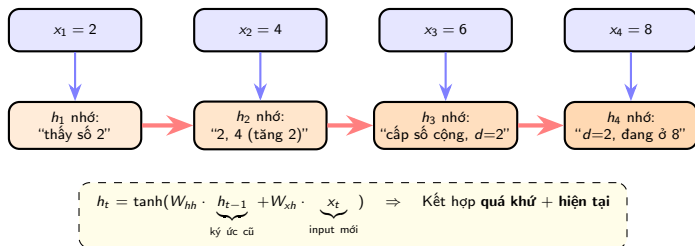
Sau huấn luyện



RNN học “công sai” của chuỗi

Qua huấn luyện, W_{hh} và W_{xh} được điều chỉnh sao cho h_t tích lũy thông tin về **xu hướng tăng** (công sai d), giúp dự đoán $x_6 = x_5 + d$.

Hidden state lưu thông tin như thế nào?



Cơ chế nhớ:

- 1 h_{t-1} : mang thông tin **tất cả các bước trước** (ký ức)
- 2 x_t : thông tin **bước hiện tại** (quan sát mới)
- 3 $\tanh(\cdot)$: nén kết hợp vào $(-1, +1)$, tạo h_t mới
- 4 h_T (bước cuối): "bản tóm tắt" toàn bộ chuỗi $\rightarrow$ dùng để dự đoán

Tóm tắt

Khái niệm	Trong ví dụ PyTorch
Input shape	(batch=2, seq_len=5, input_size=1)
Hidden state h_0	<code>torch.zeros(1, 2, 8)</code> — khởi tạo = 0
Output shape	(2, 5, 8) — h_t tại mọi bước
Dự đoán	Lấy <code>output[:, -1, :]</code> qua <code>nn.Linear(8, 1)</code>
RNN học gì?	Công sai d của chuỗi cấp số cộng
Hidden state nhớ gì?	Tích lũy xu hướng tăng/giảm qua các bước
Huấn luyện	BPTT + Adam + Gradient Clipping
Kết quả	Sai số < 0.2 trên chuỗi chưa thấy bao giờ

Bài tập mở rộng

- 1 Thay `nn.RNN` bằng `nn.LSTM` — so sánh kết quả
- 2 Tăng `hidden_size` lên 32 — model học nhanh hơn?
- 3 Thử chuỗi phức tạp hơn: $x_n = x_{n-1} + x_{n-2}$ (Fibonacci)

Questions?